



## MODUL PEMBELAJARAN

Mata Kuliah Pengantar Data Science  
Program Studi PJJ Informatika

---

# PERTEMUAN 2

Struktur Data Python,  
NumPy & Pandas

|               |                              |
|---------------|------------------------------|
| Mata Kuliah   | Pengantar Data Science       |
| Kode          | 200302305                    |
| SKS           | 3 SKS                        |
| Pertemuan ke- | 2 (Kedua)                    |
| Koordinator   | Syahid Abdullah, S.Si, M.Kom |

---

Dokumen ini merupakan panduan resmi perkuliahan untuk mahasiswa dan dosen.

# DAFTAR ISI

---

|                                                    |   |
|----------------------------------------------------|---|
| 1. Identitas dan Gambaran Umum Modul.....          | 1 |
| 1.1 Capaian Pembelajaran .....                     | 1 |
| 1.2 Prasyarat .....                                | 1 |
| 2. Struktur Data Python .....                      | 1 |
| 2.1 Pengantar .....                                | 1 |
| 2.2 List .....                                     | 2 |
| 2.3 Tuple.....                                     | 2 |
| 2.4 Dictionary.....                                | 2 |
| 2.5 Set .....                                      | 3 |
| 2.6 Perbandingan Struktur Data .....               | 3 |
| 3.1 Pengenalan NumPy .....                         | 3 |
| 3.2 Membuat NumPy Array .....                      | 4 |
| 3.3 Atribut dan Operasi Dasar .....                | 4 |
| 3.4 Operasi Vektorisasi dan Fungsi Statistik ..... | 5 |
| 4. Pandas: Manipulasi DataFrame .....              | 5 |
| 4.1 Pengenalan Pandas .....                        | 5 |
| 4.2 Pandas Series .....                            | 5 |
| 4.3 Pandas DataFrame.....                          | 6 |
| 4.4 Memuat Data CSV.....                           | 6 |
| 4.5 Filtering dan Seleksi Data .....               | 7 |
| 4.6 Groupby dan Agregasi.....                      | 7 |
| 5. Panduan Aktivitas Hands-on dan Kuis .....       | 8 |
| 5.1 Persiapan .....                                | 8 |
| 5.2 Langkah-langkah Praktikum .....                | 8 |
| 5.3 Pertanyaan Latihan.....                        | 9 |
| 6. Referensi.....                                  | 9 |

# 1. Identitas dan Gambaran Umum Modul

Modul pertemuan kedua ini merupakan kelanjutan dari Pertemuan 1. Mahasiswa akan memperdalam kemampuan pemrograman Python dengan mempelajari struktur data bawaan Python, komputasi numerik menggunakan library NumPy, serta manipulasi dan analisis data tabel menggunakan library Pandas. Ketiga topik ini merupakan fondasi praktis yang akan digunakan di seluruh mata kuliah.

## 1.1 Capaian Pembelajaran

Setelah menyelesaikan pertemuan ini, mahasiswa diharapkan mampu:

- Membedakan dan memilih struktur data Python (list, tuple, dictionary, set) yang tepat untuk berbagai kebutuhan manipulasi data.
- Membuat dan memanipulasi NumPy ndarray, serta menerapkan operasi vektorisasi dan fungsi statistik numerik.
- Membuat objek Pandas Series dan DataFrame, baik secara manual maupun dari sumber data eksternal (file CSV).
- Menerapkan teknik filtering data menggunakan boolean indexing, `.loc[]`, `.iloc[]`, dan `query()`.
- Melakukan agregasi data menggunakan operasi `groupby()` dan fungsi-fungsi `agg()` di Pandas.
- Membaca dan menganalisis dataset CSV publik secara mandiri untuk menjawab pertanyaan analitik.

## 1.2 Prasyarat

Untuk mengikuti pertemuan ini secara efektif, mahasiswa harus:

- Telah menyelesaikan seluruh aktivitas Pertemuan 1 (memiliki akun Google Colab & GitHub aktif).
- Memahami konsep dasar Python: variabel, tipe data, operator, dan fungsi dasar.
- Terbiasa membuka dan menjalankan notebook di Google Colab.

# 2. Struktur Data Python

## 2.1 Pengantar

Struktur data adalah cara mengorganisasikan dan menyimpan data dalam memori komputer agar dapat diakses dan dimodifikasi secara efisien. Python menyediakan empat struktur data bawaan (built-in) yang paling sering digunakan: list, tuple, dictionary, dan set. Pemilihan struktur data yang tepat sangat berpengaruh pada keterbacaan, efisiensi, dan kebenaran kode.

**Konsep Kunci:**

Sebelum memilih struktur data, ada tiga hal yang perlu diperhatikan: (1) Apakah urutan data penting? (2) Apakah data perlu diubah? (3) Apakah nilai duplikat diperbolehkan? Jawaban terhadap ketiga pertanyaan ini menentukan struktur data mana yang paling sesuai.

## 2.2 List

List adalah koleksi berurutan (ordered) dan dapat diubah (mutable) yang menjadi struktur data paling fleksibel di Python. List mampu menyimpan elemen dari berbagai tipe data sekaligus, mendukung duplikasi nilai, dan menyediakan berbagai metode bawaan untuk manipulasi elemen.

```
Python — List Operations
# Membuat list
data = [10, 20, 'Python', True, 3.14]

# Akses & slicing
print(data[0])      # → 10
print(data[-1])     # → 3.14 (dari belakang)
print(data[1:3])    # → [20, 'Python']

# Manipulasi
data.append(99)      # tambah di akhir
data.insert(1, 55)   # sisipkan di posisi 1
data.remove(20)      # hapus nilai pertama = 20
data.sort()          # urutkan (jika tipe sama)
```

Kompleksitas waktu akses elemen list adalah  $O(1)$  (konstan), sedangkan pencarian elemen adalah  $O(n)$  (linear). Untuk dataset besar, NumPy ndarray lebih efisien dari list Python standar.

## 2.3 Tuple

Tuple mirip dengan list, namun bersifat immutable. Artinya sekali dibuat, isinya tidak dapat diubah. Properti ini menjadikan tuple lebih cepat, lebih hemat memori, dan dapat digunakan sebagai kunci dictionary (karena hashable). Tuple ideal untuk merepresentasikan data yang tidak boleh berubah, seperti koordinat geografis, RGB color, atau pasangan kunci-nilai tetap.

**Kapan Menggunakan Tuple?**

Gunakan tuple ketika data bersifat tetap (immutable), seperti koordinat (lat, lon), tanggal (year, month, day), atau konfigurasi sistem. Gunakan list ketika data perlu dimodifikasi secara dinamis.

## 2.4 Dictionary

Dictionary adalah koleksi pasangan kunci-nilai (key-value pairs) yang tidak berurutan (Python  $\geq 3.7$  mempertahankan insertion order). Kunci harus bersifat immutable dan unik, sementara nilai dapat berupa objek apapun. Dictionary adalah fondasi dari banyak struktur data di ekosistem Python, termasuk objek DataFrame Pandas.

```

Python — Dictionary Operations
mahasiswa = {
    'nama': 'Budi Santoso',
    'nim': '2024001',
    'prodi': 'Informatika',
    'ipk': 3.85
}

# Akses nilai
print(mahasiswa['nama'])      # → 'Budi Santoso'
print(mahasiswa.get('usia', 0)) # default jika key tak ada

# Update dan hapus
mahasiswa['angkatan'] = 2024  # tambah key baru
del mahasiswa['nim']           # hapus key

```

## 2.5 Set

Set adalah koleksi tak berurutan (unordered) yang hanya menyimpan nilai unik — duplikat otomatis dihapus. Set mendukung operasi himpunan matematika: union (`|`), intersection (`&`), difference (`-`), dan symmetric difference (`^`). Set sangat efisien untuk deduplikasi data dan pengecekan keanggotaan (membership test) dengan kompleksitas  $O(1)$ .

## 2.6 Perbandingan Struktur Data

| Kriteria               | List                  | Tuple               | Dictionary        | Set                    |
|------------------------|-----------------------|---------------------|-------------------|------------------------|
| <b>Mutable?</b>        | Ya                    | Tidak               | Ya (nilai)        | Ya                     |
| <b>Berurutan?</b>      | Ya                    | Ya                  | Ya ( $\geq$ v3.7) | Tidak                  |
| <b>Duplikat?</b>       | Ya                    | Ya                  | Key: tidak        | Tidak                  |
| <b>Indexing?</b>       | Ya                    | Ya                  | Via key           | Tidak                  |
| <b>Performa akses</b>  | $O(1)$                | $O(1)$              | $O(1)$            | $O(1)$                 |
| <b>Penggunaan umum</b> | Daftar data fleksibel | Data tetap / config | Key-value mapping | Deduplikasi himpunan / |

# 3. NumPy: Komputasi Numerik

## 3.1 Pengenalan NumPy

NumPy (Numerical Python) adalah library fundamental untuk komputasi ilmiah di Python. Objek utama NumPy adalah ndarray (N-dimensional array), yaitu array multidimensi yang homogen (semua elemen bertipe data sama). NumPy diimplementasikan dalam bahasa C, sehingga 10 hingga 100 kali lebih cepat dari list Python untuk operasi numerik.

**Referensi Ilmiah:**

Harris et al. (2020) dalam jurnal Nature (585, 357–362) mendokumentasikan bahwa NumPy telah menjadi fondasi dari hampir seluruh ekosistem sains data Python, termasuk Pandas, Scikit-learn, SciPy, dan Matplotlib. Makalah ini mendapat lebih dari 10.000 kutipan dalam 3 tahun pertama.

## 3.2 Membuat NumPy Array

Terdapat beberapa cara membuat NumPy ndarray, tergantung pada kebutuhan data:

```
Python — Membuat NumPy Array
import numpy as np

# Dari list Python
a1D = np.array([1, 2, 3, 4, 5])           # 1D
a2D = np.array([[1,2,3],[4,5,6]])         # 2D

# Array dengan nilai inisial
zeros = np.zeros((3, 4))                 # nol semua
ones  = np.ones((2, 3))                  # satu semua
eye   = np.eye(4)                        # matriks identitas
full  = np.full((2, 2), 7)               # nilai konstan

# Sequence
arange = np.arange(0, 20, 2)             # 0,2,4,...18
linspace = np.linspace(0, 1, 11)         # 11 titik merata

# Array acak (reproducible)
rng = np.random.default_rng(seed=42)
rand_int = rng.integers(0, 100, (3, 4))
rand_flt = rng.normal(loc=0, scale=1, size=(100,))
```

## 3.3 Atribut dan Operasi Dasar

Setiap ndarray memiliki atribut yang memberikan informasi tentang struktur dan tipe datanya:

| Atribut             | Nilai Contoh | Deskripsi                            |
|---------------------|--------------|--------------------------------------|
| <b>arr.ndim</b>     | 2            | Jumlah dimensi (1D, 2D, 3D...)       |
| <b>arr.shape</b>    | (3, 4)       | Ukuran per dimensi: 3 baris, 4 kolom |
| <b>arr.size</b>     | 12           | Total jumlah elemen                  |
| <b>arr.dtype</b>    | float64      | Tipe data elemen                     |
| <b>arr.itemsize</b> | 8            | Ukuran memori per elemen (byte)      |
| <b>arr.T</b>        | (4, 3)       | Transpose array (balik dimensi)      |

## 3.4 Operasi Vektorisasi dan Fungsi Statistik

Salah satu keunggulan utama NumPy adalah kemampuan vektorisasi, yaitu operasi matematika diterapkan pada seluruh elemen array secara bersamaan tanpa perlu loop eksplisit. Ini menghasilkan kode yang lebih singkat, mudah dibaca, dan jauh lebih cepat.

```
Python — NumPy Vektorisasi & Statistik
# Operasi aritmetika vektorisasi
a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])
print(a + b)      # → [11 22 33 44 55]
print(a * b)      # → [10 40 90 160 250]
print(a ** 2)     # → [1 4 9 16 25]

# Universal functions (ufuncs)
np.sqrt(a)        # akar kuadrat tiap elemen
np.exp(a)         # eksponensial (e^x)
np.log(b)         # logaritma natural

# Fungsi statistik
data = rng.normal(0, 1, 1000)
print(f'Mean: {np.mean(data):.4f}')
print(f'Std: {np.std(data):.4f}')
print(f'Min: {np.min(data):.4f}')
print(f'Max: {np.max(data):.4f}')
print(f'Q1: {np.percentile(data, 25):.4f}')
print(f'Q3: {np.percentile(data, 75):.4f}')
```

## 4. Pandas: Manipulasi DataFrame

### 4.1 Pengenalan Pandas

Pandas adalah library open-source yang menyediakan struktur data dan alat analisis data berkecepatan tinggi dan mudah digunakan untuk Python. Pandas dibangun di atas NumPy dan sangat terinspirasi oleh bahasa R, khususnya konsep DataFrame-nya. Library ini pertama kali dikembangkan oleh Wes McKinney pada tahun 2008 dan kini menjadi salah satu library Python paling banyak diunduh di dunia.

#### Referensi Ilmiah:

McKinney (2010) mempresentasikan Pandas dalam makalah 'Data Structures for Statistical Computing in Python' di Proceedings of the 9th Python in Science Conference. Makalah ini mendeskripsikan desain Series dan DataFrame sebagai solusi untuk keterbatasan NumPy dalam menangani data berlabel dan heterogen.

### 4.2 Pandas Series

Series adalah struktur data satu dimensi (1D) berlabel yang mampu menyimpan data dari berbagai tipe: integer, float, string, objek Python, dan lainnya. Setiap elemen Series memiliki label yang disebut index. Series dapat dianggap sebagai kamus yang terurut, atau sebagai satu kolom dalam spreadsheet.

```
Python — Pandas Series
import pandas as pd

# Series dari list — index default 0,1,2...
s1 = pd.Series([85, 92, 78, 95, 88])

# Series dengan index kustom
nilai = pd.Series(
    [85, 92, 78, 95],
    index=['Alice', 'Bob', 'Caca', 'Dina']
)
print(nilai['Bob'])      # → 92
print(nilai[nilai>80])   # filter
print(nilai.mean())     # → 87.5
print(nilai.describe()) # statistik lengkap
```

### 4.3 Pandas DataFrame

DataFrame adalah struktur data dua dimensi berlabel, seperti tabel spreadsheet atau tabel SQL, yang terdiri dari baris (records) dan kolom (fields). Setiap kolom dalam DataFrame adalah sebuah Series. DataFrame mendukung tipe data yang berbeda per kolom, yang membedakannya dari NumPy 2D array yang hanya mendukung satu tipe data.

```
Python — Membuat Pandas DataFrame
# Membuat DataFrame dari dictionary
data = {
    'nama': ['Ali', 'Budi', 'Caca', 'Dina'],
    'usia': [22, 25, 21, 23],
    'kota': ['Jakarta', 'Bandung', 'Surabaya', 'Medan'],
    'ipk': [3.8, 3.5, 3.9, 3.7]
}
df = pd.DataFrame(data)

# Informasi dasar
print(df.shape)      # → (4, 4)
print(df.dtypes)     # tipe data per kolom
print(df.columns)    # nama-nama kolom
```

### 4.4 Memuat Data CSV

Dalam praktik Data Science, data hampir selalu dibaca dari sumber eksternal. Fungsi `pd.read_csv()` adalah cara paling umum untuk memuat data tabular ke dalam DataFrame. Fungsi ini mendukung berbagai format file, berbagai encoding, dan berbagai opsi parsing.

```
Python — pd.read_csv()
# Memuat dari file lokal
df = pd.read_csv('data/penjualan.csv')

# Memuat dari URL (dataset publik)
url = "https://raw.githubusercontent.com/..."
df = pd.read_csv(url)
```



```
# Parameter penting
df = pd.read_csv('data.csv',
    sep=';', # pemisah kolom (default ',')
    encoding='utf-8', # encoding file
    index_col='ID', # kolom sebagai index
    parse_dates=['tgl'], # parse kolom tanggal
    na_values=['N/A', 'none'] # nilai kosong
)

# Eksplorasi standar setelah load
df.head() # 5 baris pertama
df.info() # tipe data & missing
df.describe() # statistik deskriptif
```

## 4.5 Filtering dan Seleksi Data

Filtering adalah operasi paling sering dilakukan dalam analisis data, yaitu memilih subset baris atau kolom berdasarkan kondisi tertentu. Pandas menyediakan beberapa mekanisme filtering yang saling melengkapi.

### Boolean Indexing

Cara paling mendasar dan paling banyak digunakan untuk filtering di Pandas. Ekspresi kondisi menghasilkan Series bertipe boolean (True/False), yang kemudian digunakan untuk memilih baris.

```
Python — Boolean Indexing
# Contoh: Dataset Titanic
df = pd.read_csv(titanic_url)

# Filter satu kondisi
penumpang_selamat = df[df['Survived'] == 1]
penumpang_wanita = df[df['Sex'] == 'female']

# Filter kondisi ganda (& = AND, | = OR)
wanita_kelas1 = df[(df['Sex']=='female') &
    (df['Pclass']==1)]

# Seleksi kolom tertentu
df[['Name', 'Age', 'Sex', 'Survived']]
```

### .loc[] dan .iloc[]

.loc[] digunakan untuk seleksi berbasis label (nama baris/kolom), sedangkan .iloc[] berbasis posisi integer. Keduanya mendukung slicing dua dimensi: [baris, kolom].

## 4.6 Groupby dan Agregasi

Operasi groupby() memungkinkan pengelompokan data berdasarkan nilai kolom tertentu, kemudian aplikasi fungsi agregasi pada setiap grup. Ini setara dengan klausa GROUP BY dalam SQL dan operasi pivot table di Excel.

```
Python — Groupby & Agregasi
# Pola dasar: groupby → aggregation
df.groupby('Pclass')['Survived'].mean()

# Agregasi multiple kolom & fungsi
hasil = df.groupby('Pclass').agg(
    total      = ('PassengerId', 'count'),
    selamat    = ('Survived', 'sum'),
    pct_selamat= ('Survived', 'mean'),
    rata_usia  = ('Age', 'mean'),
    max_fare   = ('Fare', 'max')
)
hasil = hasil.round(3)
print(hasil)

# GroupBy multi-kolom
df.groupby(['Pclass', 'Sex'])['Survived']\
    .mean().unstack().round(3)
```

## 5. Panduan Aktivitas Hands-on dan Kuis

### 5.1 Persiapan

Sebelum aktivitas hands-on dimulai, setiap mahasiswa:

- Membuka Google Colab dan membuat notebook baru dengan nama 'Pertemuan2\_[Nama]\_[NIM].ipynb'
- Memiliki koneksi internet yang stabil untuk mengunduh dataset CSV dari URL
- Telah mengimport library yang dibutuhkan: `import pandas as pd`, `import numpy as np`

#### **Dataset yang Digunakan:**

Dataset Titanic adalah dataset klasik yang sangat populer dalam komunitas Data Science. Dataset ini berisi informasi tentang 891 penumpang kapal Titanic yang tenggelam tahun 1912, termasuk apakah mereka selamat atau tidak. Dataset ini tersedia secara bebas di Kaggle, GitHub, dan berbagai platform open data lainnya.

### 5.2 Langkah-langkah Praktikum

1. Buat notebook baru di Google Colab: 'Pertemuan2\_[Nama]\_[NIM].ipynb'
2. Salin dan jalankan semua kode pada modul ini. Amati dan pelajari output yang dihasilkan.
3. Buat section baru untuk menjalankan perintah pada nomor-nomor berikutnya.
4. Impor library: `import pandas as pd` dan `import numpy as np`
5. Muat dataset Titanic dari URL GitHub:  
<https://github.com/datasciencedojo/datasets/blob/master/titanic.csv> menggunakan `pd.read_csv()`

6. Jawab semua Pertanyaan Latihan (Q1–Q6) dalam sel terpisah, disertai komentar penjelasan
7. Tambahkan sel Markdown di antara setiap jawaban untuk menjelaskan temuan Anda
8. Jalankan semua sel dan pastikan tidak ada error sebelum menyimpan
9. Upload notebook (.ipynb) dari Google Colab ke repositori GitHub data-science anda masing-masing dan salin linknya pada Forum Diskusi 2 di LMS. Latihan ini bersifat opsional.

### 5.3 Pertanyaan Latihan

| No.       | Pertanyaan                                                                                                               |
|-----------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Q1</b> | Berapa jumlah baris dan kolom pada dataset Titanic? Kolom apa saja yang tersedia?                                        |
| <b>Q2</b> | Kolom mana saja yang memiliki missing values? Berapa jumlah dan proporsinya (%) per kolom?                               |
| <b>Q3</b> | Berapa persen penumpang yang selamat (Survived=1)? Gunakan operasi NumPy atau Pandas.                                    |
| <b>Q4</b> | Filter penumpang wanita dari kelas 1 (Pclass=1). Berapa jumlahnya dan berapa rata-rata usianya?                          |
| <b>Q5</b> | Gunakan groupby untuk menghitung tingkat keselamatan (mean Survived) per kelas penumpang. Kelas mana yang paling tinggi? |
| <b>Q6</b> | Buat analisis tambahan berdasarkan variabel lain yang Anda anggap menarik. Sertakan interpretasi singkat.                |

## 6. Referensi

- McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
- VanderPlas, J. (2016). Python data science handbook: Essential tools for working with data. O'Reilly Media. (Tersedia gratis di: <https://jakevdp.github.io/PythonDataScienceHandbook/>)
- Bruce, P., Bruce, A., & Gedeck, P. (2020). Practical statistics for data scientists: 50+ essential concepts using R and Python (2nd ed.). O'Reilly Media.
- Grus, J. (2019). Data science from scratch: First principles with Python (2nd ed.). O'Reilly Media.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference (SciPy 2010), 445, 51–56.
- Van der Walt, S., Colbert, S. C., & Varoquaux, G. (2011). The NumPy array: A structure for efficient numerical computation. Computing in Science & Engineering, 13(2), 22–30. <https://doi.org/10.1109/MCSE.2011.37>

- Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23.  
<https://doi.org/10.18637/jss.v059.i10>
- Pandas Development Team. (2020). *pandas-dev/pandas: Pandas*. Zenodo.  
<https://doi.org/10.5281/zenodo.3509134>
- NumPy Development Team. (2024). NumPy user guide and reference. <https://numpy.org/doc/>
- Pandas Development Team. (2024). *pandas documentation*. <https://pandas.pydata.org/docs/>
- Project Jupyter. (2024). Jupyter Notebook documentation. <https://jupyter-notebook.readthedocs.io/>
- Kaggle. (2024). Titanic — Machine Learning from Disaster (Dataset).  
<https://www.kaggle.com/c/titanic/data>
- Waskom, M. (2024). Seaborn: Statistical data visualization. Dataset repository:  
<https://github.com/mwaskom/seaborn-data>
- 

Dokumen ini dibuat untuk keperluan pembelajaran akademik. Hak cipta konten merujuk pada referensi masing-masing.